



CSI247: Data Structures

Lecture 01 – Course Introduction

Department of Computer Science
B. Gopolang (247-275)

Learning Objectives

In this lecture we will discuss:

- Course objectives, overview
 - The various topics to be covered in the course
 - Prescribed Textbook
- Course Admin
 - Pre-requisites
 - Timetable
 - Lab registration
 - Assessments
 - Programming environment
- Introduction to Data Structures (DS)

1. Course Objectives

- In this course, you will:
 - Learn about and implement some of the common data structures.
 - Understand costs and benefits of each data structure.

2. Course Learning outcomes

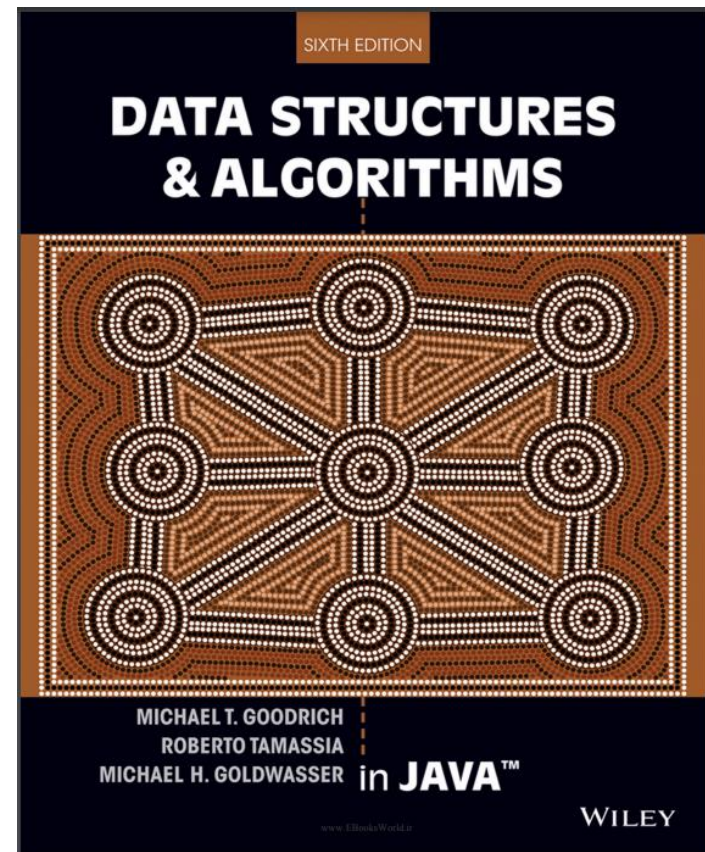
- After completion of the course, you should be able to:
 1. Describe common applications for each data structure discussed in the course
 2. Implement these user-defined data structures using a high-level language (Java in this case)
 3. Write programs using each discussed data structures
 4. Choose an appropriate data structure for a given problem
 5. Perform worst-case runtime analysis on data structures and simple algorithms

3. Course outline

- Topics will include, but not limited to, the following:
 - Review of Java programming basics
 - Recursion
 - Time complexity
 - Searching and sorting algorithms
 - Using arrays
 - Object oriented programming
 - Implementation and use of common Data Structures:
 - Stacks & Queues: using arrays and linked lists
 - Trees, Graphs & hash tables (time permitting)

4. Course material

- Lecture notes
 - Handouts (pdfs) on Moodle
- Text book:
 - “Data Structures & Algorithms in Java” by Goodrich, Tamassia and Goldwasser, 6th Ed, 2014



5. Pre-requisites

- Courses:
 - CSI142 (& CSI141) or ISS112: $\geq 50\%$
- **Required basic knowledge:**
 - Java Programming basics
 - Programming structures (loops & selection)
 - File I/O
 - User-defined classes, objects & methods
 - Arrays

6. Timetable

- **Lectures**
 - M, W & F: 7am – 8am, 252/001
- **Lab session**
 - R, 9-11am or 11-1pm
 - Lab registration: online (CS Moodle)
 - Changes NOT ALLOWED
 - Room allocation: after lab registration
- **Attendance**
 - Compulsory for both lectures and labs!

7. Assessments

- Final mark:
 - CA (50%) + Ex (50%)
 - a) CA:
 - 4 quizzes (1% + 3% + 3% + 3%)
 - 2 Tests (40%)
 - Date, time and venue for tests and quizzes: TBA
 - b) Final Exam: UB Exam timetable will apply
- Important:
 - ALWAYS bring **your valid UB student ID** to these

8. Useful tools/resources

- a) CS Moodle
 - Handouts (pdfs). Print and bring to class
 - Labs
 - Announcements
- b) Visual Studio Code:
 - Programming environment
 - Available in allocated CS labs
- c) WhatsApp:
 - For quick and easy communication
 - Join via link provided on CS Moodle

Introduction to Data Structures

Introduction

- Recap: A computer program?
 - Implementation of an algorithm
 - Well-defined set of instructions for solving a specific problem, in a finite amount of time
 - Implemented using a programming language

- What happens during algorithm design?
 - Determine the correct logic - technique(s) to apply
 - Is that all?

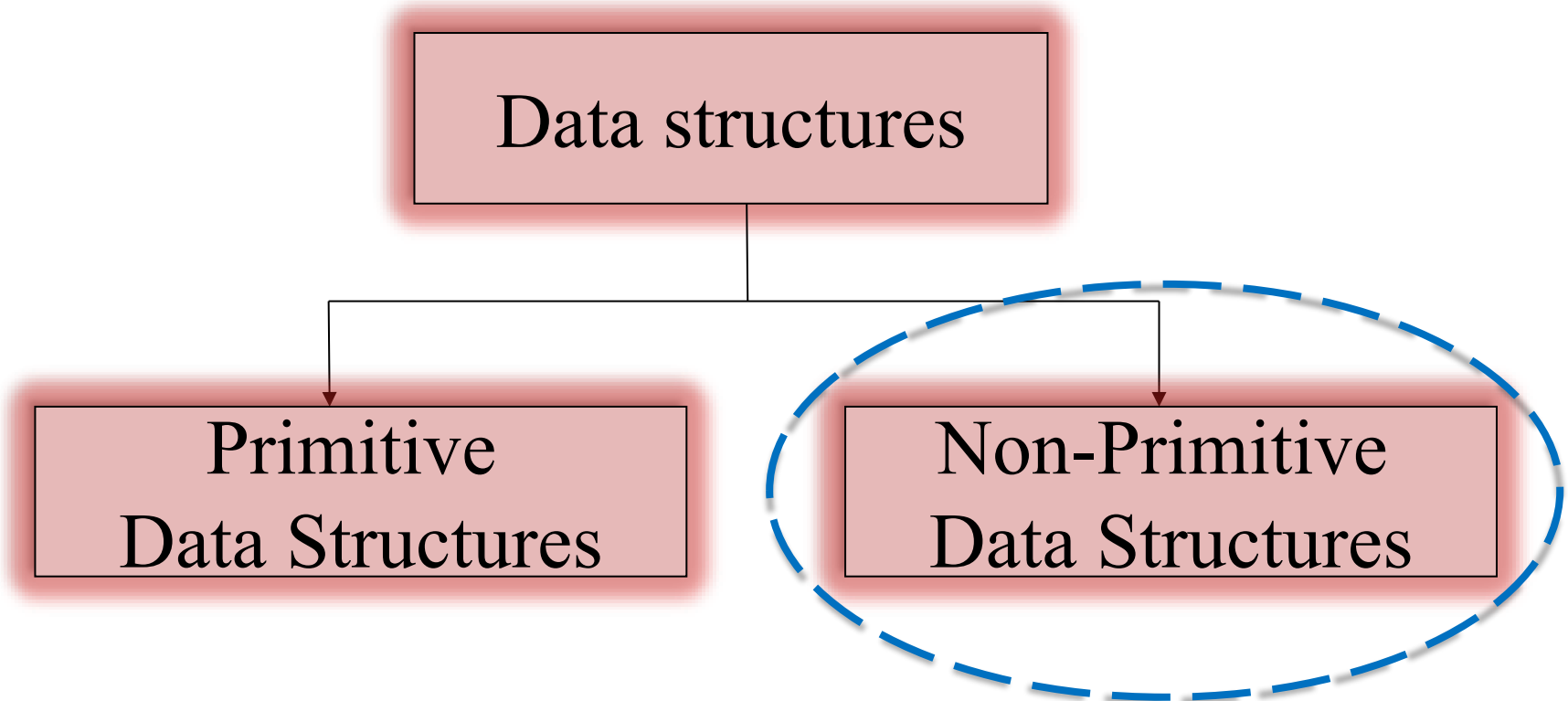
- Answer:
 - No,
 - An algorithm alone IS NOT ENOUGH to solve a problem
 - You also need an appropriate data structure
- So, program is:

Algorithm + Data Structure

- What is a Data Structure (DS)?
 - A way of **organizing data** in a computer's memory
 - Presents a program's data manipulation framework
 - Specifies possible operations on stored data
 - Different types available
 - E.g.: array, stack, etc (later !)
 - Note:
 - Choice of data structure affect program design
 - Determines an algorithm's performance

Classification of Data Structures

- 2 broad categories:



a) Primitive data structures:

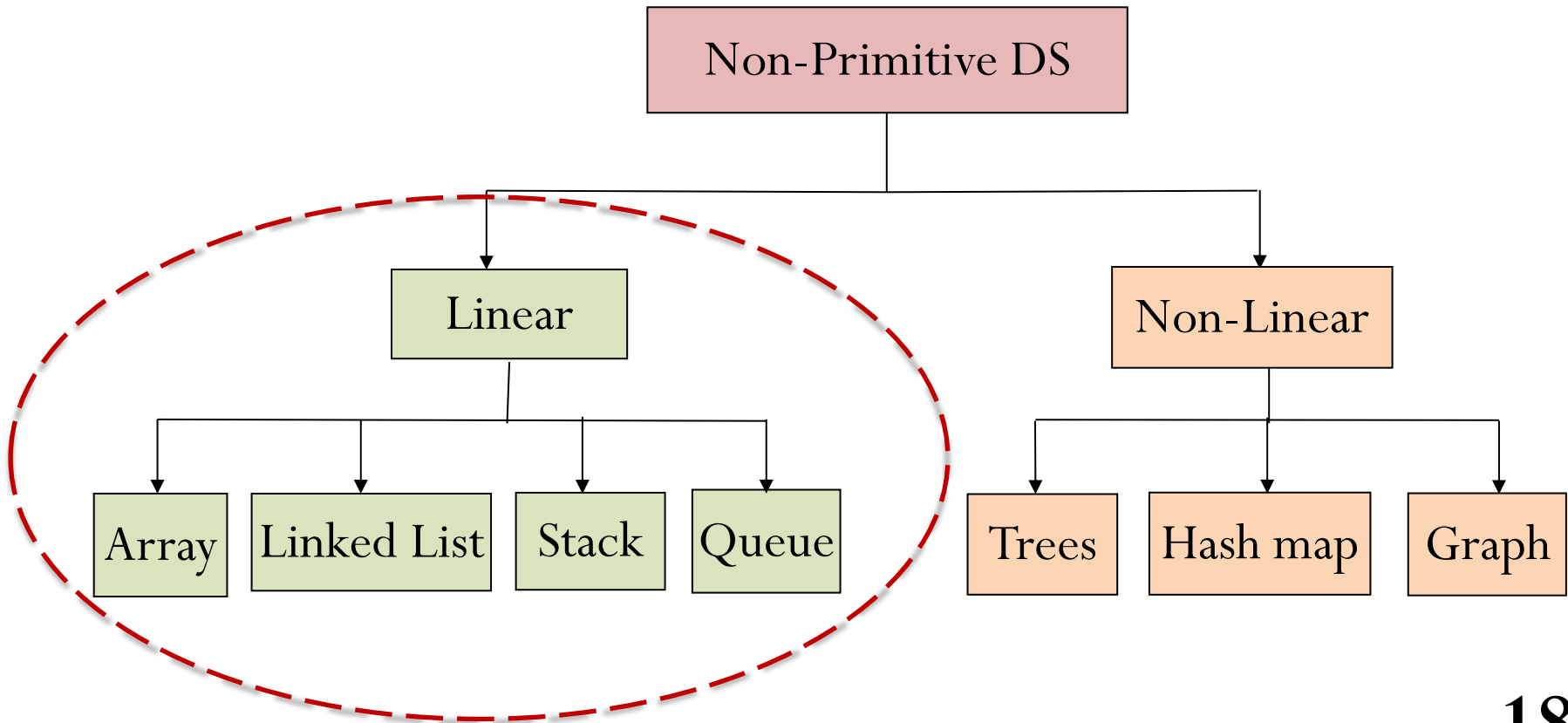
- **Basic structures**

- We used them in 1st year programming course(s)
- Examples:
 - integers
 - characters
 - Etc

b) Non-primitive data structures:

- **More complex structures**
 - Derived from primitive data structures
- Different classifications:
 - Homogeneous vs heterogeneous
 - Type of data items stored
 - Linear vs non-linear
 - Placement of data items
 - Static vs dynamic
 - Etc.

- Further classification: linear vs. non-linear DSs



Common DS Operations

- Various activities can be performed on a DS:
 - Create a DS
 - Add/insert an item
 - Delete an item
 - Modify/change/update
 - Search - locate a specific item
 - Sort - order items
 - Merge
 - etc

Why study DS?

- Rem: Program processes data (I, P & O)
 - Data must be stored somewhere in memory
 - It must be organized properly
 - For **efficient** storage and retrieval
- Key qualities of the most appropriate (efficient) DS:
 - Offers easy storage, access and modification
 - Quick access to data
 - Uses less memory space.

DS application areas

- Quite many:
 - OS – process scheduling, e.g. printing (queues), dynamic memory allocation (linkedlists), etc
 - Graphics – arrays, LinkedLists, trees, etc
 - AI and machine learning – quick retrieval of items
 - Arrays, LinkedLists, Hash Tables, Trees, etc
 - Simulation – queues, heaps, etc
 - Internet (e.g. stack for Back button, etc)
 - DBMS – indexing data items
 - Data analytics
 - Etc

Summary

- Course objectives, overview
- Topics to cover in the course
- Course material
- Course admin: schedule, pre-requisite, labs
- Assessments
- Useful tools
- Introduction to Data Structures

Next Lesson

- Review: Java programming basics

Q & A



Thank you